Wireless Deauthentication Detection Dashboard

Project Overview:

The Wireless Deauthentication Detection Dashboard is a wireless intrusion detection system designed to monitor WiFi networks and identify potential Deauthentication attacks. The system captures 802.11 management frames in monitor mode, analyses disconnect patterns, and alerts administrators when suspicious activity is detected. It also provides a graphical dashboard for real-time monitoring and logs events for forensic analysis.

This project demonstrates packet-level wireless analysis, anomaly detection, and real-time visualization for WiFi security monitoring.

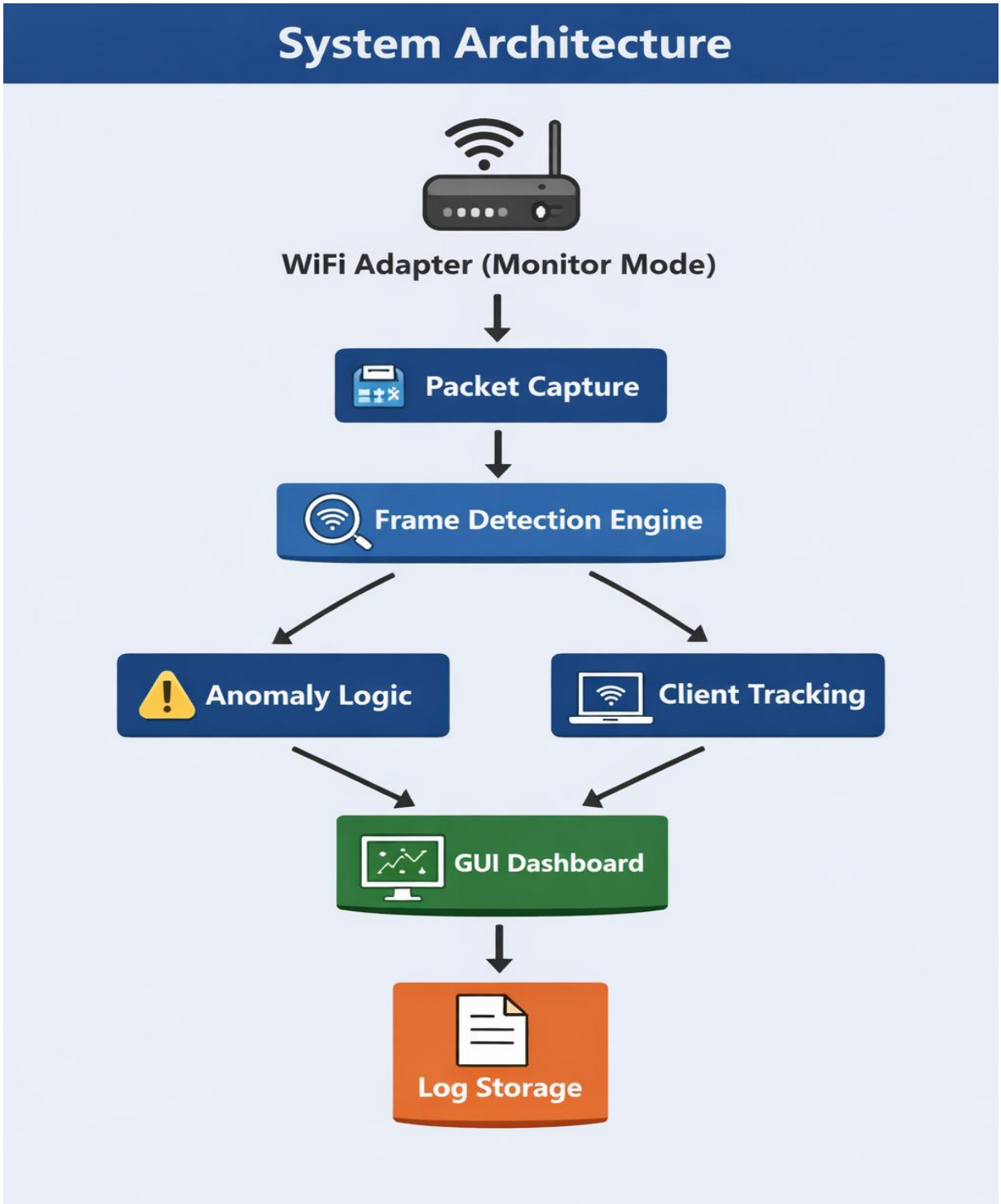
Objectives:

- Capture 802.11 wireless frames in monitor mode
- Detect deauthentication and disassociation frames
- Identify abnormal disconnect patterns
- Track affected clients
- Provide real-time GUI dashboard
- Visualize attack activity
- Log events for forensic analysis
- Demonstrate WPA3/PMF awareness

Technologies Used:

- Python 3
- Scapy
- Matplotlib
- Aircrack-ng
- Linux (Ubuntu)
- 802.11 Monitor Mode

System Architecture:



Features:

- Real-time wireless packet capture
- Deauthentication detection
- Disassociation detection
- Anomaly-based attack detection
- Client impact tracking
- GUI dashboard visualization
- Real-time event logging
- WPA3 / WiFi 6 compatibility awareness

Requirements:

- Ubuntu Linux
- Python 3
- WiFi adapter supporting monitor mode
- Aircrack-ng
- Scapy
- Matplotlib

Installation:

sudo apt update

sudo apt install aircrack-ng python3-venv python3-full

Create environment:

mkdir wifi-ids

cd wifi-ids

python3 -m venv venv

source venv/bin/activate

pip install scapy matplotlib

Running the Project:

Enable monitor mode:

```
sudo airmon-ng check kill

sudo airmon-ng start wlp2s0
```

Lock channel:

```
sudo ip link set wlp2s0mon down

sudo iw dev wlp2s0mon set channel 149

sudo ip link set wlp2s0mon up
```

Run dashboard:

```
sudo ./venv/bin/python gui_logging_detector.py
```

How It Works:

1. The WiFi adapter is switched to monitor mode.
2. Scapy captures raw 802.11 frames.
3. The system filters deauthentication and disassociation frames.
4. Anomaly detection identifies repeated disconnect patterns.
5. Alerts are displayed on GUI dashboard.
6. Events are logged to file for analysis.

Output:

The dashboard displays:

- Real-time disconnect events
- Alert counter
- Attacker MAC address
- Target client MAC address
- Timestamp of event

Example Log:

[2026-03-22 15:40:11] ALERT

Attacker: a4:ce:da:30:d7:13

Client: 48:55:5e:03:1d:d2

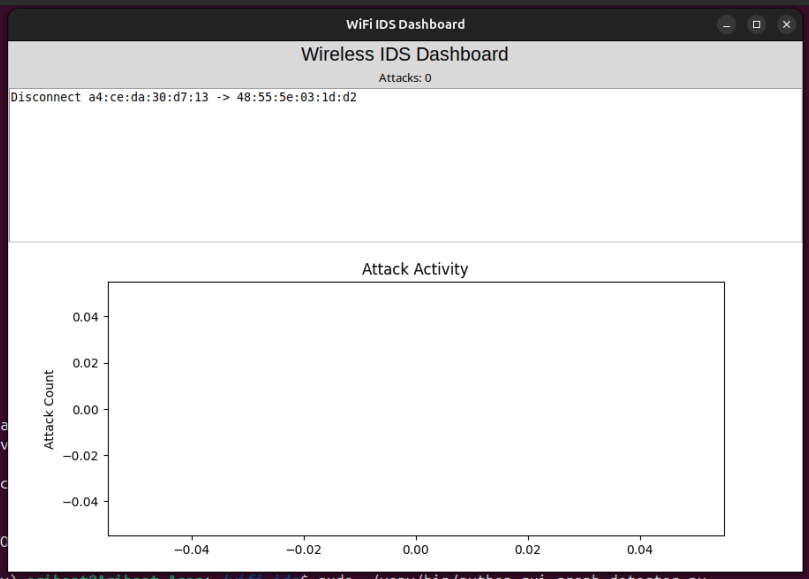
```
arihant@Arihant-Acer: ~/wifi-ids
arihant@Arihant-Acer:~$ cd wifi-ids/
arihant@Arihant-Acer:~/wifi-ids$ source venv/bin/activate
(venv) arihant@Arihant-Acer:~/wifi-ids$ pip install matplotlib
Requirement already satisfied: matplotlib in ./venv/lib/python3.12/site-packages (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./venv/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.12/site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in ./venv/lib/python3.12/site-packages (from matplotlib) (2.4.3)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.12/site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.12/site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.12/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in ./venv/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
(venv) arihant@Arihant-Acer:~/wifi-ids$ sudo airmon-ng check kill
[sudo] password for arihant:

Killing these processes:

    PID Name
  19744 wpa_supplicant
  20363 avahi-daemon
  20373 avahi-daemon
```

```
(venv) arihant@Arihant-Acer:~/wifi-ids$ sudo airmon-ng start wlp2s0

Found 2 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

    PID Name
  20382 avahi-daemon
  20385 avahi-daemon

PHY     Interface       Driver          Chipset

phy0    wlp2s0          rtw88_8822ce    Realtek Semiconductor Co., Ltd. RTL8822CE 802.11ac PCIe Wireless Network Adapter
                (mac80211 monitor mode vif enabled for [phy0]wlp2s0 on [phy0]wlp2s0mon)
                (mac80211 station mode vif disabled for [phy0]wlp2s0)

(venv) arihant@Arihant-Acer:~/wifi-ids$ iwconfig
lo        no wireless extensions.

enp3s0f1  no wireless extensions.

wlp2s0mon  IEEE 802.11  Mode:Monitor  Frequency:2.457 GHz  Tx-Power=20 dBm
          Retry short limit:7   RTS thr:off   Fragment thr:off
          Power Management:on
```

```
(venv) arihant@Arihant-Acer:~/wifi-ids$ ls
advanced_detector.py  deauth_detector.py  gui_detector.py  gui_graph_detector.py  sniff_test.py  venv
(venv) arihant@Arihant-Acer:~/wifi-ids$
(venv) arihant@Arihant-Acer:~/wifi-ids$ sudo ./venv/bin/python deauth_detector.py
Starting Deauth Detector...

🚨 Deauthentication Frame Detected!
Time : 2026-03-22 14:10:21.845542
Source: a4:ce:da:30:d7:13
Target: 48:55:5e:03:1d:d2
BSSID : 48:55:5e:03:1d:d2
--------------------------------------

🚨 Deauthentication Frame Detected!
Time : 2026-03-22 14:10:44.134457
Source: a4:ce:da:30:d7:13
Target: 48:55:5e:03:1d:d2
BSSID : 48:55:5e:03:1d:d2
--------------------------------------
```

```
Mar 22  14:11                                                    98%
arihant@Arihant-Acer: ~/wifi-ids
  GNU nano 7.2                                   advanced_detector.py
from scapy.all import *
from scapy.layers.dot11 import Dot11Deauth, Dot11Disas
from collections import defaultdict
import datetime
import time

tracker = defaultdict(list)

THRESHOLD = 2
WINDOW = 5

def detect(pkt):

    if pkt.haslayer(Dot11Deauth) or pkt.haslayer(Dot11Disas):

        src = pkt.addr2
        dst = pkt.addr1
        now = time.time()

        tracker[src].append(now)

        tracker[src] = [t for t in tracker[src] if now - t < WINDOW]

        print("\n⚠️Disconnect frame detected")
        print("Source:", src)
        print("Target:", dst)

                                   [ Read 36 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location  M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

```
Mar 22  14:14                                                    98%
arihant@Arihant-Acer: ~/wifi-ids
```




```
Comma
'wdev

You c
e.g.

Do NO
(venv) arihant@Arihant-Acer:~/wifi-ids$ sudo ./venv/bin/python gui_graph_detector.py
```

Testing Method:

The system was tested using:

- Windows laptop as client
- Mobile phone as client
- WiFi toggle to generate disconnect events
- WPA3 WiFi 6 network environment

Limitations:

- WPA3 Protected Management Frames reduce visible deauth frames
- Some devices disconnect silently
- Requires monitor mode supported adapter

Future Enhancements:

- Web dashboard interface
- Prometheus integration
- Email alerting
- Multi-graph analytics
- Attacker localization using RSSI

Conclusion:

This project successfully demonstrates a wireless intrusion detection system capable of monitoring WiFi networks and detecting suspicious disconnect behaviour. It provides real-time visualization, logging, and analysis of wireless management frames, making it useful for WiFi security monitoring and research.

Demo Commands:

```
cd ~/wifi-ids

source venv/bin/activate

sudo airmon-ng check kill

sudo airmon-ng start wlp2s0

sudo ip link set wlp2s0mon down

sudo iw dev wlp2s0mon set channel 149

sudo ip link set wlp2s0mon up

sudo ./venv/bin/python gui_logging_detector.py
```